Instances of the lonely runner conjecture for integer speeds in $[1, 2m]$

Vico Bonfioli

9 November 2026. Preprint, not peer reviewed.

Abstract

For a positive integer m let $V \subset [1, 2m]$ be a set of m distinct positive integers. We consider the inequality $\mathrm{ML}(V) \geq 1/(m+1)$, the lonely runner conjecture for m speeds restricted to speeds of size at most $2m$. For every $m \geq 2$ such that $2m+1$ is prime it follows from a divisibility lemma of Tao, a witness device of Bohman and Peng used at modulus $2m+1$, and one forcing step (Theorem 3); the Lean declaration is archived with one axiom print of 2026-10-01. For composite $2m+1$ the inequality is established by finite certificates checked by the Lean 4 kernel, for all 73 values $m \leq 126$: the four rows of Table 1 (fractional blocking), the 68 rows of Table 2 (cover capacity), and $m = 94$, which is closed by a tree of three pinned cover-capacity certificates branching on whether 67 and 75 are speeds (Section 8). For each of the 73 values a raw axiom print of the Lean declaration, listing only `propext`, `Classical.choice` and `Quot.sound`, is stored in the accompanying archive, and the modules contain no `sorry`. The Lean builds were run in the development repository and were not repeated for this archive. Consequently the statement is VERIFIED, in the sense of Section 1, for every m with $2 \leq m \leq 126$. Nothing is claimed for $m \geq 127$ except when $2m+1$ is prime, and the question for all m (Problem 6 of the survey of Perarnau and Serra) is not settled. The covers were found by computer searches of several kinds (Table 2, search column) and were accepted by a second, independent Rust checker before the Lean build. The window has height $2m$, whereas the known reductions of the conjecture require exponential height, so nothing here bears on the conjecture for all m. This is a special case of the lonely runner conjecture, by a single author, and it has not been refereed.

1 Setting, labels and results

Write $\|x\| = \mathrm{dist}(x, \mathbb{Z})$. For $v = (v_1, \ldots, v_m) \in \mathbb{Z}^m_{>0}$ put $\mathrm{gap}_v(t) = \min_i \|v_i t\|$ and $\mathrm{ML}(v) = \sup_{t \in \mathbb{R}} \mathrm{gap}_v(t)$. The lonely runner conjecture (LRC) for m speeds is $\mathrm{ML}(v) \geq 1/(m+1)$ for all v; there are $m+1$ runners, one of them stationary; a title of the form "n runners" in the cited literature means $n-1$ speeds. Throughout, m counts speeds. For a finite set V of positive integers write $\mathrm{ML}(V) = \mathrm{ML}(v)$ for any v listing the elements of V (repetitions do not change gap_v). The *floor* is the value $1/(m+1)$. A time $t \in \mathbb{R}$ is a *witness* for v if $\mathrm{gap}_v(t) \geq 1/(m+1)$; since gap_v is continuous and periodic, $\mathrm{ML}(v) \geq 1/(m+1)$ if and only if v has a witness. The *dense window* statement for m is: $\mathrm{ML}(v) \geq 1/(m+1)$ whenever the v_i are distinct and lie in $[1, 2m]$. Tao names the window $[1, 2m]$ as a natural target [13]; it is Problem 6 of the survey [7]. In Lean, `gap` and `ML` are as above for `Fin (k+1) → ℤ` with $m = k+1$ (module `MaxLoneliness`).

Labels. VERIFIED: formalized in Lean 4 with Mathlib, build clean, no `sorry`, axioms among `propext`, `Classical.choice`, `Quot.sound`. PROVED: a complete written proof, not formalized.

COMPUTED: an exact finite computation on the stated range, with no general proof. HEURISTIC: plausibility or a computation on a sample of instances only. CONJECTURE: no proof. A result has the weakest label in its dependency chain. Where the project record contains no axiom print for a declaration, this is stated. The mark S (Table 2, column "axioms"; the four rows of Table 1 and the three leaves of Table 3 are all S) refers to the print `#print axioms` of the Lean declaration named in the row: it lists exactly `propext, Classical.choice, Quot.sound`, which excludes `sorryAx` and `Lean.ofReduceBool`. The print is stored as a raw file in the archive (file named per row in Section 7). For 21 rows ($m = 24, 25, 27, 28, 31, 32, 34, 37, 38, 40, 42, 43, 45, 46, 47, 55$ and $m = 97, 103, 109, 115, 122$) the stored print is the line of a results file of a guarded single-module development build of 2026-11-03, with the local path inside one error message rewritten. The declaration printed is `dense_floor_m`m (monolithic form) for $m \leq 62$, `dense_floor_m`m`_split` (split form, Remark 11) for $m \geq 64$ with $m \neq 94$, and `dense_floor_m94` for $m = 94$; for $m = 62$ both of the first two are printed. The print is evidence only for the declaration named; the module build status is evidence only for the module. All builds and prints are those of the development repository and are carried over to the archive as stored files; no Lean build was run to prepare this release. The archived sources of the modules differ from the files of the development tree only in comments (paths in docstrings were rewritten; comment-stripped comparison of 2026-11-09). A reader holding only this PDF can open none of these prints.

Results.

- Theorem 3: dense window for $2m + 1$ prime, with a written proof (Lemmas 1–4) and the exact checks of Section 3. Lean declaration: axiom print of the 2026-10-01 development run archived (`data/dense/dev_logs/thmD_axioms.out`), module not rebuilt for this text, statement matched to the paper statement by reading only; the written proof (which has a different form for Lemma 4) was reviewed and is PROVED; the exact checks cover $m \leq 12$ only. Label: VERIFIED for the declaration on the strength of that one archived print; PROVED for the written argument.

- Proposition 4: fractional blocking criterion; instances $m = 4, 7, 12, 13$. VERIFIED.

- Theorem 5: cover-capacity criterion. VERIFIED.

- Theorem 10: dense window for the composite values of $2m + 1$ listed in Tables 1 and 2. The listed rows number 72 (Table 1: 4; Table 2: 68, of which 29 with $m \leq 62$ and 39 with $m \geq 64$), all marked S: the raw axiom print is archived (for 21 of them it is the line of a development results file of 2026-11-03, see the paragraph on labels). Label: VERIFIED for each of the 72 rows in the sense of Section 1.

- Proposition 12 (pinned cover capacity) and Theorem 13 (dense window for $m = 94$, $2m + 1 = 189$), Section 8: the one composite $2m + 1 \leq 253$ without a row in Table 2. Lean declarations `floor_of_pinned_cover` and `dense_floor_m94`; VERIFIED. An exhaustive computer search for the same statement (hitting sets of the 3,724 minimal bad sets) is reported as corroboration and is COMPUTED; the Lean proof does not use it.

- Propositions 6–9: structure used by the certificate search. Proposition 6 is a computational observation with no proof for general m (HEURISTIC); Proposition 7 is a consequence of it, with a short written proof given in Section 6 that has not been independently reviewed (HEURISTIC, since it rests on Proposition 6); Propositions 8 and 9 are PROVED. None of them is used by Theorems 3–10, whose proofs depend only on the Lean-checked certificates.

Section 10 lists what is not claimed. In particular Theorem 3 is a routine consequence of known ingredients (Section 11).

Statement of significance. Tao [13] describes the bound $1/(m+1)$ for speeds in $[1, 1.2m]$ (the proof is in the accompanying preprint [12], as described in his blog post) and names the range $[1, 2m]$ as an interesting sub-problem; he notes that several tight tuples, among them $(1, \dots, m)$, its double and $(1, 2, 3, 4, 5, 7, 12)$, lie in that range. To our knowledge no paper proves the bound for all sets of m distinct speeds in $[1, 2m]$, either in general or for an infinite family of m. Theorem 3 does this for all m with $2m + 1$ prime, and Theorems 10 and 13 for all $m \leq 126$. The basis for "to our knowledge" and its limits are stated in Section 11 (paragraph "Search of 2026-11-09"): in particular the papers of Bohman and Peng, Pomerance and Pandey were not re-read in that search, and what this paper says about them has not been re-checked against Theorem 3. The result is a special case of the lonely runner conjecture, by a single author, and has not been refereed.

Provenance. The results were obtained with the assistance of an AI system (Claude, Anthropic). The questions and research directions came from the author. The written proofs, the programs, the certificate searches and the Lean sources were produced by the AI system; the certificates are checked by the Lean kernel and by an independent Rust checker, and the written arguments and the Lean statements were read by further sessions of the same system. The author is responsible for all claims.

2 Outline of the proof

This section lists the steps by which the dense window statement is proved, with the result each step uses and one sentence on why it suffices. Later sections prove the steps; none of them is used here beyond its statement. Recall that t is a witness for v if $\mathrm{gap}_v(t) \geq 1/(m+1)$. The statement for m is that every v with entries in $[1, 2m]$ has a witness.

Prime $2m + 1$ *(Section 3, Theorem 3).*

P1. If 1 is not a speed, then $t = 1/(2m + 2)$ is a witness (Lemma 2). Every speed lies in $[2, 2m]$ and so stays at distance at least $2/(2m + 2)$ from the integers.

P2. If some $q \in [2, m + 1]$ divides no speed, then $t = 1/q$ is a witness (Lemma 1). Each speed has a nonzero residue modulo q, hence $\|v_i/q\| \geq 1/q \geq 1/(m + 1)$.

P3. Otherwise 1 is a speed and every $q \in [2, m + 1]$ divides a speed; then two speeds sum to $2m + 1$ (Lemma 3). The speeds $m + 1$ (the only multiple of $m + 1$ in $[1, 2m]$) and m or $2m$ (the multiples of m) are forced; either $m + (m + 1) = 2m + 1$ or $1 + 2m = 2m + 1$.

P4. If $N = 2m + 1$ is prime and two speeds sum to N, some time c/N is a witness (Lemma 4). The m classes $\{s, N - s\}$ partition $[1, 2m]$, the speeds meet at most $m - 1$ of them, and the inverse modulo N of an unmet s is a multiplier c that keeps every residue $v_i c$ away from 0 and ± 1.

Steps P1 to P3 hold for every $m \geq 2$. Only P4 needs primality.

Composite $2m+1$ *(Sections 4 to 7).* Here P4 fails (Section 3, paragraph on primality), so witnesses at other denominators are certified by finite data.

C1. For a time $t = p/q$ let $B(t) \subseteq [1, 2m]$ be the set of speeds x with $\|xt\| < 1/(m+1)$. A set V without a witness meets every $B(t)$; its complement $W = [1, 2m] \setminus V$ has m elements and contains no $B(t)$ (Section 4). So it suffices to exhibit finitely many times whose sets $B(t)$ rule out every such V.

C2. Fractional blocking (Proposition 4): integer weights on finitely many times such that each x lies in $B(t)$ for times of total weight at most D, and the total weight exceeds mD. A set V of m elements meets $B(t)$ for times of total weight at most mD, so some time of positive weight has $V \cap B(t) = \emptyset$. This closes $m = 4, 7, 12, 13$ (worked example in Section 5).

C3. Cover capacity (Theorem 5) where C2 fails: cover $[1, 2m]$ by parts U_i and give each a capacity c_i such that every subset of U_i with more than c_i elements contains some $B(t)$. The set W of C1 then has at most c_i elements in U_i, so $m = |W| \leq \sum c_i$, and $\sum c_i < m$ is a contradiction.

C4. The certificates of C2 and C3 for the values of m in Tables 1 and 2 were found by computer search (Section 6 describes the structure the search uses; it is not part of the proof). Each certificate is a finite list of integers, accepted by an independent exact checker and by the Lean kernel (Theorem 10); the search is irrelevant to correctness.

C5. For $m = 94$, for which no cover of the form of C3 is known, branch on whether 67 and 75 are speeds: three pinned cover-capacity certificates (Proposition 12, Section 8) close the three cases. Together with the prime case and Theorem 10, this gives the statement for every $2 \leq m \leq 126$.

3 The prime case

Let $N = 2m + 1$ with N prime. No distinctness is needed in this section: $V = (v_1, \ldots, v_m)$ is a multiset of integers in $[1, 2m]$.

Theorem 3 (Lean declaration: one archived axiom print, module not rebuilt for this text; written proof PROVED)**.** *Let $m \geq 2$ with $2m+1$ prime, and $v_1, \ldots, v_m$ integers in $[1, 2m]$. Then* $\mathrm{ML}(v) \geq 1/(m+1)$.

Lean: `dense_floor_of_prime` (module `Delta2DenseWindow`), with $m = k + 1$, hypotheses $k \geq 1$, $0 < v_i \leq 2(k+1)$, $2k + 3$ prime. The case $m = 1$ is trivial ($t = 1/2$) and excluded only for uniformity. The statement was matched against the Lean statement by reading; the module was not rebuilt for this archive; the axiom print of the 2026-10-01 development run (`propext, Classical.choice, Quot.sound`; `data/dense/dev_logs/thmD_axioms.out`, also `thmD_chain.out`) was not repeated. The weakest link of the label is therefore: one development run of 2026-10-01 whose raw print is archived, a hand comparison of the Lean statement with the statement above, and no rebuild since. The prints a reader of this PDF alone cannot reach are all of them (the archive is separate; none is reproduced here).

Idea of the proof. Speeds that avoid 1 are beaten by the small time $1/(2m+2)$. If 1 is a speed and no time $1/q$ with $q \leq m + 1$ wins, every integer from 2 to $m + 1$ divides some speed; as all speeds are at most $2m$, this forces $m + 1$ and one of m, $2m$, hence two distinct speeds summing to the prime $N = 2m + 1$. The move is to test only times c/N. A speed v spoils such a time only if c is congruent to plus or minus the inverse of v modulo N, that is, only if v lies in the class

$\{s, N-s\}$ of $s = c^{-1}$. These m classes partition $[1, 2m]$, and two speeds summing to N lie in one class, so the speeds meet at most $m-1$ classes. The inverse of a missed class is a good multiplier. A search over real times becomes a count of classes.

Lemma 1 (Witnesses, VERIFIED)**.** *Let p, q be positive integers. If $q \le (m+1)\min(v_i p \bmod q,\ q - (v_i p \bmod q))$ for all i, then $\mathrm{gap}_v(p/q) \ge 1/(m+1)$. In particular, if some $q \in [2, m+1]$ divides no v_i, then $\mathrm{ML}(v) \ge 1/(m+1)$ (take $p = 1$).*

Lean: `cube_floor_of_witness`, `floor_of_not_dvd_le` (module `Delta2DigitGreedy`); the latter is the declaration called by `dense_floor_of_prime` in case 2. The second statement is the case used by Tao's Lemma 5.1(i) [12]: if $\mathrm{ML}(v) < 1/(m+1)$ and $v_i \le 2m$, every $j \in [2, m+1]$ divides some v_i; it is stated in Lean as `tao_5_1_i` (module `Delta2DenseWindow`).

Proof. At $t = p/q$ one has $\|v_i t\| = \min(r_i, q - r_i)/q \ge 1/(m+1)$ with $r_i = v_i p \bmod q$. For the second claim take $p = 1$: if $q \nmid v_i$ then $1 \le r_i \le q-1$, so $\min(r_i, q - r_i) \ge 1$ and $m + 1 \ge q$. □

Lemma 2 (Speeds avoiding 1)**.** *If $1 \le v_i \le 2m$ and no v_i equals 1, then $\mathrm{ML}(v) \ge 1/(m+1)$.*

Proof. Take $t = 1/(2m+2)$. Since $2 \le v_i \le 2m$, we have $v_i < 2m + 2$ and $\|v_i t\| = \min(v_i, 2m + 2 - v_i)/(2m+2) \ge 2/(2m+2) = 1/(m+1)$. □

Lean: the branch $1 \notin V$ of `dense_floor_of_prime` (via `cube_floor_of_witness`); status VERIFIED.

Lemma 3 (Forced pair)**.** *Let $m \ge 2$ and $1 \le v_i \le 2m$. Suppose that $1 \in \{v_i\}$ and every $q \in [2, m+1]$ divides some v_i. Then there are indices $a \ne b$ with $v_a + v_b = 2m + 1$.*

Proof. The multiples of $m+1$ in $[1, 2m]$ reduce to $m+1$, since $2(m+1) > 2m$; so some $v_b = m+1$. The integer m lies in $[2, m+1]$ because $m \ge 2$, and its multiples in $[1, 2m]$ are m and $2m$. If some $v_a = m$, then $v_a + v_b = 2m + 1$ and $a \ne b$ since $m \ne m + 1$. Otherwise some $v_a = 2m$, and with $v_{i_1} = 1$ we have $v_{i_1} + v_a = 2m + 1$, with $i_1 \ne a$ because $1 \ne 2m$. □

Lean: the branch of `dense_floor_of_prime` with $1 \in V$ and full resonance (the two forced divisibilities are `hb`, `ha` in the source); status VERIFIED.

Lemma 4 (Pair summing to a prime, VERIFIED statement)**.** *Let $N = 2m + 1$ be prime, $1 \le v_i \le 2m$, and suppose $v_a + v_b = N$ for some $a \ne b$. Then $\mathrm{ML}(v) \ge 1/(m+1)$.*

Proof. For $1 \le s \le m$ let $C_s = \{s, N - s\}$; these m classes partition $[1, 2m]$. Since N is odd, $v_a \ne v_b$, and both lie in one class. The set of values of the v_i has at most m elements, so it meets at most $m - 1$ classes (the distinct values v_a, v_b share a class). Choose s with C_s disjoint from $\{v_1, \ldots, v_m\}$ and let $c \in \{1, \ldots, N-1\}$ be the inverse of s modulo N, which exists since N is prime. Put $r_i = v_i c \bmod N$. Then $r_i \ne 0$, because $1 \le v_i < N$. If $r_i \equiv \pm 1$, then $v_i \equiv \pm s$, hence $v_i \in C_s$, which is excluded. Therefore $2 \le r_i \le N - 2$, and at $t = c/N$ we get $\|v_i t\| = \min(r_i, N - r_i)/N \ge 2/N > 1/(m+1)$, because $2m + 2 > 2m + 1$. □

Lean: `floor_of_pairsum_prime` (module `Delta2Pigeonhole`), whose docstring describes the equivalent multiplier count: each v_i forbids at most two of the $2m$ nonzero multipliers c, namely $\pm v_i^{-1}$, and $v_b \equiv -v_a$ gives the same two, so at most $2m - 2$ are forbidden. The two counts correspond under $c \leftrightarrow c^{-1}$. The class form is the declaration `floor_of_missed_class` (module `Delta2DenseWindow`). The written proof above was not formalized in this form. The device is that of Bohman and Peng at the sum N (Section 11).

Proof of Theorem 3. If $1 \notin \{v_i\}$, Lemma 2 applies. If some $q \in [2, m+1]$ divides no v_i, Lemma 1 applies. Otherwise 1 is a speed and every $q \in [2, m+1]$ divides a speed; Lemma 3 gives a pair summing to $2m+1$ and Lemma 4 concludes. □

Where each hypothesis is used. The bound $v_i \leq 2m$ is used in Lemmas 2, 3 and 4 (the distance $\geq 2/(2m+2)$; uniqueness of the multiple $m+1$; $N \nmid v_i$). The bound $v_i \geq 1$ is used in Lemmas 2 and 4. Primality of N is used only in Lemma 4, where it gives an inverse of s for every class C_s. Lemmas 1, 2 and 3 hold for every $m \geq 2$. Repeated speeds are harmless: every count is over values and only shrinks.

Primality. For composite $N = 2m+1$ the conclusion of Lemma 4 remains true for every instance tested, but its proof fails: there exist sets with a pair summing to N for which no time c/N is a witness. The smallest is $m = 7$, $N = 15$, $V = \{1, 2, 4, 5, 6, 7, 8\}$; here $\mathrm{ML}(V) = 2/13 > 1/8$, attained away from denominator 15, and no c in $1, \ldots, 14$ gives a witness $c/15$. Modulo 15 the set meets the classes $\{1, 14\}$, $\{2, 13\}$, $\{4, 11\}$, $\{5, 10\}$, $\{6, 9\}$, $\{7, 8\}$ and misses only $\{3, 12\}$, whose element 3 is not invertible. There are 32, 544, 368 such sets (of size at most m) for $m = 7, 10, 12$ respectively, and none for $m = 4$, $N = 9$. These sets have witnesses at other denominators.

Height. At height $2m+1$ the exact search finds no set without a witness for $m \leq 12$, so the truth of the bound does not stop at $2m$ in this range. The proof does: Lemma 2 fails for $v = 2m+1$ (distance $1/(2m+2)$ at $t = 1/(2m+2)$), and Lemma 4 fails because $N \mid v$ is possible. The searches at heights $2m+2$ and $3m$ likewise found no violating set.

Exact checks. All checks below enumerate sets of size at most m in $[1, 2m]$ (the supports of the multisets of m speeds) and use exact integer arithmetic; the witness criterion of Lemma 1 is a sufficient condition, so a reported witness is never false. Programs and outputs: `code/dense/src/thmd`, `data/dense/thmd`.

- Window $[1, 2m]$, $1 \leq m \leq 12$ (`dense_check.rs`, times c/s with $2 \leq s \leq 4m$): every set has a witness; 13,015,068 sets in total (the sum over $1 \leq m \leq 12$ of the numbers of sets of size at most m in $[1, 2m]$, namely 2, 10, 41, 162, 637, 2509, 9907, 39202, 155381, 616665, 2449867, 9740685; the saved output lists these values and the program was re-run with identical output); the search completed without a size or node cap. The number of independent cases is 12, namely the values of m, of which 8 have $2m+1$ prime and 4 have $2m+1 \in \{9, 15, 21, 25\}$. An independent Python program (`cross_check.py`, fractions, all times a/b with $b \leq 6m$) agrees for $2 \leq m \leq 8$.

- Class form of Lemma 4 (`class_check.rs`), primes $2m+1$ with $m \leq 9$ ($m = 2, 3, 5, 6, 8, 9$). The program runs over all subsets of $[1, 2m]$ of size at most m (10, 41, 637, 2509, 39202, 155381 of them respectively), not only those containing a pair with sum N; of these, 2, 15, 395, 1781, 32642, 135699 contain two elements with sum N. For each of them it checks that the subset misses a class, and that the time c/N with $c = s^{-1}$ for the least missed s is a witness; 0 exceptions. Six independent cases. Negative control: the subsets with no such pair that meet all m classes admit no c with all residues outside $\{0, \pm 1\}$. The prime $N = 23$ ($m = 11$) was not run through this program; for $m = 11$ only the case counts below bear on it (there the witness test runs over all times c/N, and no case-3 set lacks a witness).

- Case counts of the table below (`dense_check.rs steps`; recounted by `case_recount.rs` for $4 \le m \le 12$ with identical numbers): in every instance Lemma 3 held, and each of the three witness times was verified on the sets of its case. For composite N the counts of case-3 sets without a witness c/N are those of the paragraph on primality.
- Controls: the programs return $\mathrm{ML}(1,2,3,4,5) = 1/6$, $\mathrm{ML}(1,3,4,7) = 1/5$, $\mathrm{ML}(1) = 1/2$, $\mathrm{ML}(1,2) = 1/3$, and, as a control on the witness test, no witness for $\{1,3,4,7\}$ at the threshold $1/4$. For $m = 4$ the floor is $1/5$, and $\mathrm{ML}(1,3,4,7) = 1/5$ equals it, attained at $t = 1/5$ (recomputed with the standard exact routine); the threshold $1/4$ exceeds ML, so a reported witness there would signal an error. The same search at heights $2m+1$ ($m \le 12$), $2m+2$ ($4 \le m \le 11$) and $3m$ ($3 \le m \le 8$) finds a witness for every set.

The programs share the definition of a witness and so are independent in code only. No $m > 12$ was checked; beyond $m \le 12$ the statement rests on the proof. For $m \le 12$ it is also a consequence of the cited proofs of the conjecture, and for $m = 13, 14$ of [1], which is unrefereed. These computations are not part of the proof of Theorem 3.

m	$2m+1$	case 1	case 2	case 3	case 3 without witness $c/(2m+1)$
5	11	381	232	24	0
6	13	1485	934	90	0
8	17	22818	15578	806	0
9	19	89845	61050	4486	0
11	23	1401291	1007773	40803	0
4	9	98	60	4	0
7	15	5811	3830	266	32
10	21	354521	251554	10590	544
12	25	5546381	4046534	147770	368

Numbers of sets of size at most m in $[1, 2m]$ by case (unnumbered table): case 1, 1 absent; case 2, 1 present and some $q \in [2, m+1]$ divides no element; case 3, the remaining sets. The upper block has $2m+1$ prime, the lower block composite.

4 Certificates for composite $2m+1$

A velocity set $V \subset [1, 2m]$ *has no witness* if for every time $t = p/q$ some $x \in V$ lies in the bad set $B(p/q) = \{x \in [1, 2m] : \|xp/q\| < 1/(m+1)\}$. Thus V is a transversal (a set meeting every member) of the family of bad sets. By Lemma 1, a witness gives the floor, so it suffices to show that no m-set is a transversal. In Lean, `goodAt k x p q` is the Boolean $q \le (k+2)\min(xp \bmod q,\ q - xp \bmod q)$, and certificate times are given as pairs (p_j, q_j) with a finite set `bad j` that contains every $x \in [1, 2m]$ failing `goodAt` at (p_j, q_j).

Fractional blocking. For a transversal V of size m and weights $y_j \ge 0$ on the times with $\sum_j y_j\,[x \in B_j] \le D$ for every $x \in [1, 2m]$ (the left side is the *load* of x, written $\mathrm{load}(x)$),

$$\sum_j y_j \le \sum_j y_j\,|B_j \cap V| = \sum_{x \in V} \mathrm{load}(x) \le mD.$$

Hence $\sum_j y_j > mD$ excludes transversals of size m. This is weak LP duality for the covering number of the hypergraph, scaled to natural numbers. The principle that an arbitrary weighting

of times, that is a finitely supported measure, may be tested against a family is Bedert's Lemma 2.1 [3], and his Lemma 5.3 and Corollary 5.4 use the uniform measure on the times j/p for a prime p dividing no speed; the weights above are an instance of the former.

Proposition 4 (VERIFIED)**.** *Let $k+1=m$, let $y:\{1,\dots,r\}\to\mathbb{N}$ be weights on r times (in the Lean source r is written `N`; it is unrelated to $N=2m+1$ of Section 3), $D\in\mathbb{N}$, and assume* $\mathrm{load}(x)\le D$ *for every $x\in[1,2m]$ and $mD<\sum_j y_j$. Then every v with $0<v_i\le 2m$ satisfies* $\mathrm{ML}(v)\ge 1/(m+1)$*. In particular, no distinctness is needed.*

Lean: `floor_of_frac_blocking` (module `Delta2FracBlocking`). Instances `dense_floor_m4`, `m7`, `m12`, `m13` (Table 1).

m	$2m+1$	times	load bound D	$\sum y_j/D$
4	3^2	5	1	$5>4$
7	$3\cdot 5$	9	2	$15/2>7$
12	5^2	15	2	$25/2>12$
13	3^3	21	11	$145/11>13$

Table 1: Fractional blocking certificates. Times: number of times with positive integer weight; the weights are integers y_j and D is the bound on the load, so the normalised total weight is $\sum_j y_j/D$, to be compared with m (Proposition 4). The columns are regenerated by the exact-integer program `code/dense/src/fb`, which reads the weights out of the Lean source of `Delta2FracBlocking`, checks that every load is at most D and that the total weight exceeds mD, and rejects four perturbed certificates (`data/dense/fb_check.out`, `fb_selftest.out`). For $2m+1=21,33,49$ the fractional bound equals m and cannot close the case (docstring of `Delta2CoverCapacity`); this statement is quoted and was not re-derived. All four rows are marked S (Section 1): the print for `dense_floor_m4`, `m7`, `m12`, `m13` is in `data/dense/dev_logs/thmD_axioms.out`.

Cover capacity. Let $W=[1,2m]\setminus V$, so $|W|=m$. If V has no witness then W is *independent*: it contains no bad set B_j (otherwise the time p_j/q_j would be a witness for V, since $V\cap B_j=\emptyset$). Let $U_1,\dots,U_t$ cover $[1,2m]$ and let c_i be such that every $T\subseteq U_i$ with $|T|>c_i$ contains some B_j. Then

$$m=|W|\le\sum_i|W\cap U_i|\le\sum_i c_i.$$

So $\sum_i c_i<m$ is a contradiction. The capacities act as cuts that the linear program does not use. For $2m+1=21$ the cover-capacity certificate of Section 5 shows that no transversal has size at most $m=10$, whereas the fractional bound is 10 (quoted, not re-derived). The weighting $1/2$ on each of the 18 velocities other than 1 and 11, together with weight 1 on each of the two singleton bad sets' velocities 1 and 11, is a fractional transversal of total weight $2+18\cdot(1/2)=11$, not 10; so it is not an optimal solution of that linear program, and the optimal weighting is not uniform.

Theorem 5 (VERIFIED)**.** *Let $v:\mathrm{Fin}(k+1)\to\mathbb{Z}$, $m=k+1$, be positive and injective with values in $[1,2m]$. Suppose finitely many times (p_j,q_j) with sets $\mathtt{bad}_j$ as above, parts $U_i\subseteq\mathbb{N}$ with $[1,2m]\subseteq\bigcup_i U_i$, capacities c_i such that every $T\subseteq U_i$ with $|T|>c_i$ contains some $\mathtt{bad}_j$, and $\sum_i c_i<m$. Then* $\mathrm{ML}(v)\ge 1/(m+1)$.

Lean: `floor_of_cover_capacity` (module `Delta2CoverCapacity`). Parts may overlap. The Lean declaration assumes injectivity, which it uses to count $|W| = m$. The written argument above does not need it: if the values of v are not distinct, the set V of values has fewer than m elements and $|W| \geq m$, and $m \leq |W| \leq \sum_i c_i$ still contradicts $\sum_i c_i < m$. The non-injective form is therefore PROVED here but not formalized in that form; nothing in Theorem 10 uses it.

Zero slack. Let $\mathrm{cap}(S)$ be the largest size of a subset of S containing no minimal bad set. By Proposition 6 below (HEURISTIC, per m), every cover has $\sum \mathrm{cap}(U_i) \geq m - 1$ for the relevant m. A certificate therefore has $\sum c_i = m - 1$ exactly, which is what every row of Table 2 has.

5 Worked examples

This section first runs the prime-case argument of Section 3 on one set ($m = 5$, $N = 11$), and then shows the two certificate kinds of Section 4 proving the statement for one composite $2m + 1$ each, $m = 4$ ($2m + 1 = 9$) and $m = 10$ ($2m + 1 = 21$). Examples 1 and 2 use only the definitions of Section 4. Their numbers were recomputed in exact integer arithmetic by a short Rust program (`code/dense/src/examples/ex.rs`); the numbers of the prime example were recomputed for this version by a short integer program (awk, not archived) and are small enough to check by hand. In Examples 1 and 2 the sets $B(p/q)$ below are exactly the $x \in [1, 2m]$ with $q > (m + 1)\min(xp \bmod q,\ q - xp \bmod q)$.

Example P: the prime case, $m = 5$, $N = 11$. The floor is $1/6$ and speeds lie in $[1, 10]$. Take $V = \{1, 2, 4, 5, 6\}$. Step P1 does not apply, since $1 \in V$. Step P2 does not apply: each $q \in \{2, 3, 4, 5, 6\}$ divides a speed ($2 \mid 2, 4, 6$; $3 \mid 6$; $4 \mid 4$; $5 \mid 5$; $6 \mid 6$). So Lemma 3 applies, and indeed $5 + 6 = 11$. The classes $C_s = \{s, 11 - s\}$ are $\{1, 10\}, \{2, 9\}, \{3, 8\}, \{4, 7\}, \{5, 6\}$. The set meets $\{1, 10\}$ (by 1), $\{2, 9\}$ (by 2), $\{4, 7\}$ (by 4) and $\{5, 6\}$ (by 5 and 6), so it meets four of the five classes, and misses only $\{3, 8\}$. Take $s = 3$; its inverse modulo 11 is $c = 4$ because $3 \cdot 4 = 12 \equiv 1$. The residues $v\,c \bmod 11$ for $v = 1, 2, 4, 5, 6$ are $4, 8, 5, 9, 2$, none of them 0 or ± 1 (that is 1 or 10). At $t = 4/11$ the distances $\|vt\| = \min(r, 11 - r)/11$ are $4/11$, $3/11$, $5/11$, $2/11$, $2/11$; the least is $2/11 \approx 0.1818 > 1/6 \approx 0.1667$. So $t = 4/11$ is a witness. Checking all ten multipliers $c = 1, \ldots, 10$ in the same way shows that exactly $c = 4$ and $c = 7 = -4$ are witnesses for this set. A different set meeting all classes but one would use that class in the same way; a set meeting all five classes cannot contain a pair summing to 11 (the pair would share a class).

Example 1: $m = 4$, fractional blocking. The floor is $1/5$ and speeds lie in $[1, 8]$. Take the five times of the certificate `dense_floor_m4`, each with weight $y = 1$:

time p/q	$1/5$	$1/10$	$1/4$	$1/3$	$4/9$
$B(p/q)$	$\{5\}$	$\{1\}$	$\{4, 8\}$	$\{3, 6\}$	$\{2, 7\}$

For $1/5$ the distances $\|x/5\|$, $x = 1, \ldots, 8$, are $1/5,\ 2/5,\ 2/5,\ 1/5,\ 0,\ 1/5,\ 2/5,\ 2/5$, so only $x = 5$ falls below $1/5$. For $4/9$ one has $8/9$ and $28/9 = 3 + 1/9$ for $x = 2, 7$, at distance $1/9 < 1/5$, and the other six values of x give distance at least $1/5$. The five sets are pairwise disjoint and cover $[1, 8]$, so every x lies in exactly one of them: the load of every x is 1, $D = 1$, and the total weight is $5 > mD = 4$. Now let V be any four elements of $[1, 8]$. Each element lies in one set $B(p/q)$, so V meets at most four of the five sets, and some time p/q in the list has $V \cap B(p/q) = \emptyset$. By Lemma 1 this time is a witness. This is Proposition 4. For $V = \{1, 3, 4, 7\}$ the elements lie in $B(1/10)$,

$B(1/3)$, $B(1/4)$, $B(4/9)$ and the free time is $1/5$: the distances $\|v/5\|$ are $1/5,\ 2/5,\ 1/5,\ 2/5$. (The program also confirmed that each of the 70 four-element subsets of $[1,8]$ (the binomial coefficient $\binom{8}{4}$) has a free time; the least number of free times over the subsets is 1.) Only the time $4/9$ has denominator $2m+1=9$; the certificate needs the denominators $5, 10, 4, 3$ as well.

Example 2: $m = 10$, cover capacity. The floor is $1/11$ and speeds lie in $[1, 20]$. The certificate `dense_floor_m10` uses 14 times, which are the entries of the table, and five parts:

p/q	$B(p/q)$	p/q	$B(p/q)$
$1/22$	$\{1\}$	$1/7$	$\{7, 14\}$
$1/11$	$\{11\}$	$7/22$	$\{3, 19\}$
$1/8$	$\{8, 16\}$	$9/28$	$\{3, 6\}$
$5/21$	$\{4, 17\}$	$5/31$	$\{6, 12, 19\}$
$12/25$	$\{2, 4\}$	$1/9$	$\{9, 18\}$
$15/32$	$\{2, 15, 17\}$	$1/10$	$\{10, 20\}$
$7/18$	$\{5, 13, 18\}$	$5/22$	$\{9, 13\}$

part U_i	capacity c_i	sets $B(p/q)$ inside U_i
$\{1\}$	0	$\{1\}$
$\{11\}$	0	$\{11\}$
$\{2, 4, 8, 15, 16, 17\}$	3	$\{8, 16\}, \{4, 17\}, \{2, 4\}, \{2, 15, 17\}$
$\{3, 6, 7, 12, 14, 19\}$	3	$\{7, 14\}, \{3, 19\}, \{3, 6\}, \{6, 12, 19\}$
$\{5, 9, 10, 13, 18, 20\}$	3	$\{9, 18\}, \{10, 20\}, \{5, 13, 18\}, \{9, 13\}$

The parts cover $[1, 20]$ (the sets $\{1\}, \{11\}$ and the three parts of size six are disjoint and have $2 + 18 = 20$ elements), and $\sum c_i = 9 < 10 = m$. Capacities: every subset of U_i with more than c_i elements contains some $B(p/q)$. For the parts $\{1\}$ and $\{11\}$ this holds because the part is itself a set B. For $U = \{2, 4, 8, 15, 16, 17\}$ suppose a subset S with four elements contains none of $\{8, 16\}$, $\{4, 17\}$, $\{2, 4\}$, $\{2, 15, 17\}$. It has at most one of $8, 16$ and at most one of $4, 17$, so $2, 15 \in S$ and exactly one of each pair is present; $2 \in S$ excludes 4, so $17 \in S$, and then $\{2, 15, 17\} \subseteq S$, a contradiction. The other two parts have the same shape (a pair $\{a, a'\}$, a pair $\{b, b'\}$, a pair linking one element of the second pair to a free element, and a triple), and the same argument applies with $\{14, 7\}, \{3, 19\}, \{3, 6\}, \{6, 12, 19\}$ and with $\{9, 18\}, \{10, 20\}, \{9, 13\}, \{5, 13, 18\}$; the program checked all 15 four-element subsets of each part (the binomial coefficient $\binom{6}{4}$) and found that the largest subset containing no $B(p/q)$ has exactly three elements. By Theorem 5: if V had no witness, $W = [1, 20] \setminus V$ would have 10 elements, contain no $B(p/q)$, and so have at most c_i elements in U_i, whence $10 \le 0 + 0 + 3 + 3 + 3 = 9$, which is false. Hence every set of ten distinct integers in $[1, 20]$ has a witness among the 14 times. As a direct check, the program tested all 184,756 ten-element subsets of $[1, 20]$ (the binomial coefficient $\binom{20}{10}$) and found none that meets all 14 sets $B(p/q)$.

The fractional weighting method cannot give this case: the fractional bound for $2m + 1 = 21$ (that is $m = 10$) equals m. This is quoted from the docstring of the Lean module and was not re-derived; it is mentioned in the caption of Table 1, but $m = 10$ is not a row of that table, whose rows are the values closed by fractional blocking; the row for $m = 10$ is in Table 2. The capacities 3 on parts of size 6 are what the linear program does not see.

6 Structure used by the search

Notation: $Y = \{m+2, \dots, 2m\}$ ($m-1$ elements), Low $= \{1, \dots, m+1\}$. Minimal bad sets are the inclusion-minimal sets among the $B(p/q)$. The velocities 1 and $m+1$ carry singleton minimal bad sets, so they are covered by a part $\{1, m+1\}$ of capacity 0 and removed; write Low$' = \{2, \dots, m\}$. These statements guide the search and give no input to Theorems 3–10.

Blocks. For $S \subseteq [1, 2m]$ let $\tau(S)$ be the least size of a set that meets every minimal bad set contained in S. A subset of S contains no minimal bad set contained in S if and only if its complement in S meets all of them; hence the largest such subset has size $\alpha(S) = |S| - \tau(S)$. A *block* is a pair (T, L) with $T \subseteq Y$ and $L \subseteq$ Low$'$; it is *valid* if $\tau(T \cup L) \geq |L|$. If Y contains no minimal bad set (Proposition 6), then L meets every minimal bad set contained in $T \cup L$, so $\tau(T \cup L) \leq |L|$, and validity reads $\tau(T \cup L) = |L|$, equivalently $\alpha(T \cup L) = |T|$. A block (T, L) is *kept* if it is valid, no $L' \supsetneq L$ makes (T, L') valid, and no $T' \subsetneq T$ makes (T', L) valid. A block (T_0, L_0) is *dominated* by (T, L) if $T \subseteq T_0$ and $L \supseteq L_0$. For $k \geq 0$ let F_k be the set of kept blocks with $|T| \leq k$. A *cover* of a family F of blocks is a finite list of blocks of F whose T-parts are pairwise disjoint and whose L-parts have union Low$'$; the L-parts may overlap. The programs enumerate F_4 without the block $(\emptyset, \emptyset)$ (last paragraph of Section 6); a valid block $(\emptyset, L)$ with $L \neq \emptyset$ would force every element of L to be a singleton minimal bad set.

Proposition 6 (HEURISTIC)**.** *For each m tested, Y contains no minimal bad set. Hence $y = \mathbf{1}_Y$ is a feasible dual, every cover satisfies $\sum_i \mathrm{cap}(U_i) \geq m - 1$, and in a cover of cost exactly $m - 1$ each Y-element is covered once and $\mathrm{cap}(U_i) = |U_i \cap Y|$.*

The first sentence is an exact integer check on the enumerated list of minimal bad sets. It was repeated for $2 \leq m \leq 62$ by the program `code/dense/src/yind`, which enumerates every bad set exactly (breakpoints and midpoints of the arrangement over all real times) and tests whether any bad set lies inside Y, and for $4 \leq m \leq 62$ by the test `y_independent` of `code/dense/src/cg` (command `cg edges M`); the two programs share no code and agree on the number of distinct bad sets, the number of minimal bad sets and the verdict at all 59 values $4 \leq m \leq 62$ (`data/dense/yind_2_62.out`, `cg_edges_4_62.out`; as a negative control `yind control` finds a bad set inside $\{m+1, \dots, 2m\}$). The search programs `g1` assert the property at run time for each $m \geq 64$. No proof for general m is known to the author; the statement is a computational observation for the m listed, and an assertion checked at run time for $m \geq 64$. Its consequences are derived in Proposition 7.

Proposition 7 (HEURISTIC)**.** *Given Proposition 6, a cover of cost $m-1$ is a packing of* blocks *(T_j, L_j) with $T_j \subseteq Y$ pairwise disjoint, $L_j \subseteq$ Low$'$ with $\bigcup L_j =$ Low$'$, and (T_j, L_j) valid, that is $\tau(T_j \cup L_j) = |L_j|$ by Proposition 6. Leftover Y-elements are singleton parts of capacity* 1*.*

Proof of Proposition 7, assuming Proposition 6 (written for this version; not independently reviewed, not formalized). Let $U_1, \dots, U_t$ be a cover of $[1, 2m]$ with capacities $c_i \geq \mathrm{cap}(U_i)$ and $\sum c_i = m - 1$. The velocities 1 and $m+1$ are singleton minimal bad sets, so they lie in no set free of bad sets; deleting them from all parts changes no $\mathrm{cap}(U_i)$, and they are covered at cost 0 by the part $\{1, m+1\}$. The remaining parts cover $Y \cup$ Low$'$. Since Y contains no minimal bad set, and every bad set contains a minimal one, each $U_i \cap Y$ is free of bad sets, so $\mathrm{cap}(U_i) \geq |U_i \cap Y|$. Therefore

$$m - 1 = \sum c_i \geq \sum \mathrm{cap}(U_i) \geq \sum |U_i \cap Y| \geq |Y| = m - 1,$$

the last inequality because the parts cover Y. All inequalities are equalities: every element of Y lies in exactly one part, so the sets $T_i = U_i \cap Y$ are pairwise disjoint, and $\mathrm{cap}(U_i) = |T_i|$. With $L_i = U_i \cap \mathrm{Low}'$ we have $\bigcup L_i = \mathrm{Low}'$ and, by $\alpha + \tau = |S|$ (Blocks paragraph), $\tau(T_i \cup L_i) = |T_i| + |L_i| - \mathrm{cap}(U_i) = |L_i|$, so each (T_i, L_i) is valid. A part with $L_i = \emptyset$ is a set $T_i \subseteq Y$ of capacity $|T_i|$, and may be replaced by singletons of capacity 1.

Validity of a block is decreasing in L and increasing in T. Proof: τ is monotone under inclusion, since the minimal bad sets contained in S are among those contained in S' when $S \subseteq S'$, so a transversal of the latter is one of the former; and $\tau(S) \le \tau(S \setminus \{x\}) + 1$, since adding x to a transversal of the minimal bad sets contained in $S \setminus \{x\}$ gives one for S. Hence for $L' \subseteq L$, $\tau(T \cup L') \ge \tau(T \cup L) - |L \setminus L'| \ge |L'|$ when (T, L) is valid, and for $T \subseteq T'$, $\tau(T' \cup L) \ge \tau(T \cup L) \ge |L|$ when (T, L) is valid. These two facts are used only to justify that the programs test keptness and build the valid L by single-element steps (`xc`, `g1`); the proofs of Propositions 8 and 9 do not use them. In the programs $L \subseteq \mathrm{Low}'$ holds because the velocities 1 and $m+1$ carry singleton minimal bad sets and so have no private minimal bad set with a Y-part. The equivalence of $\alpha(S) = |T|$ and $\tau(S) = |L|$ is the identity $\alpha + \tau = |S|$ stated above.

Proposition 8 (PROVED)**.** *Every valid block (T_0, L_0) is dominated by a kept block. If $L_0 \neq \emptyset$ the dominating block has nonempty L, so it is not $(\emptyset, \emptyset)$. Consequently, for every $k \ge 0$ and every valid block (T_0, L_0) with $|T_0| \le k$, the family F_k contains a block dominating it; there is no bound on $|L_0|$. All of this holds for every m (it is an order argument about any set of pairs). The block $(\emptyset, \emptyset)$ is valid for every m; it is kept if and only if no $(\emptyset, L)$ with $L \neq \emptyset$ is valid, that is, if* Low' *contains no singleton minimal bad set. That this is so was checked for $2 \le m \le 64$ only (last paragraph of this section), so "$(\emptyset, \emptyset)$ is kept and is the only kept block with $T = \emptyset$" is claimed for $m \le 64$ and not for $m \ge 66$. The programs omit $(\emptyset, \emptyset)$, which changes neither the statement above for $L_0 \neq \emptyset$ nor the existence of covers (the block covers nothing).*

Idea. Among the valid blocks that dominate a given block, take the one with the largest L and, among those, the smallest T. A kept block is one that cannot be improved by enlarging L or by shrinking T; an improvement would be a valid block in the same set, better for the order used in the choice. So the two closure conditions in the definition of a kept block reduce to one optimization over a finite set. The proof uses no property of validity and holds for any set of pairs.

Proof. Outline: (a) the valid blocks dominating (T_0, L_0) form a finite nonempty set D; (b) an element of D extremal in the order below is kept; (c) its T-part lies in T_0.

(a) Let D be the set of valid blocks (T, L) with $T \subseteq T_0$ and $L \supseteq L_0$. It is finite and contains (T_0, L_0). (b) Choose $(T, L) \in D$ with $|L|$ maximal and, among those, $|T|$ minimal. If $L' \supsetneq L$ and (T, L') is valid, then $(T, L') \in D$ and $|L'| > |L|$, contrary to the choice. If $T' \subsetneq T$ and (T', L) is valid, then $(T', L) \in D$ with the same $|L|$ and $|T'| < |T|$, contrary to the choice. Hence (T, L) is kept, and it dominates (T_0, L_0). (c) $T \subseteq T_0$ gives $|T| \le |T_0| \le k$, so $(T, L) \in F_k$. □

Computed check, not part of the proof (COMPUTED): the program `code/dense/src/st89` enumerates all pairs (T, L) with $|T| \le 4$, $T \subseteq Y$, $L \subseteq \mathrm{Low}'$ for each $2 \le m \le 17$, computes validity from the minimal bad sets (enumerated by its own exact arrangement over all real times; this enumeration is validated against an exact computation of ML on all m-subsets of $[1, 2m]$ for $3 \le m \le 8$ and on $\{1, \dots, m+1\}$, a transversal, and the exact computation reproduces $\mathrm{ML}(1, 2, 3, 4, 5) = 1/6$), and finds no valid block undominated by a kept block; deleting one kept block always produces undominated blocks (negative control). For $m = 13, 16, 17$ the numbers of valid pairs with $T \neq \emptyset \neq L$ agree with those logged for the earlier program (25,106; 70,412;

87,151). For $m = 13$ there are 3,252,224 pairs, 25,900 valid blocks and 4,478 kept blocks including $(\emptyset, \emptyset)$, 4,477 without it (`data/dense/st89_check.out`). This is a check of the statement on instances; the earlier brute-force test of the family produced by the search program (about 2.3×10^8 pairs for $m = 13, 16, 17$) is not repeated. The restriction $|T| \le 4$ is a restriction of the searched family, not of the problem: a failure to find a cover in the family says nothing about covers with larger T.

Proposition 9 (PROVED)**.** *Let $k \ge 1$. A block (T, L) is* reducible *if $T = T_1 \sqcup T_2$ and $L = L_1 \sqcup L_2$ with all four sets nonempty and (T_1, L_1), (T_2, L_2) both valid. Let R_k be the set of blocks of F_k that are not reducible. Then R_k has a cover if and only if F_k has a cover. Moreover the least value of $\sum |T|^2$ over covers is the same for F_k and R_k; the least number of blocks is not preserved.*

Idea. Among all covers of F_k take one for which the sum of the squares of the sizes of the T-parts is least. A reducible block in it can be replaced by two blocks that are no larger in T and no smaller in L, because each half of the split is a valid block and Proposition 8 gives a kept block dominating it. The replacement keeps the T-parts disjoint and the lows covered, and the weight falls, since $(a + b)^2 > a^2 + b^2$ for $a, b \ge 1$. So the minimal cover contains no reducible block, and the reduction loses no cover.

Proof. Outline: (a) a weight on covers; (b) replacement of a reducible block by two kept blocks of smaller T gives a cover of smaller weight; (c) a cover of least weight has no reducible block.

Since $R_k \subseteq F_k$, a cover of R_k is a cover of F_k. Conversely let C be a cover of F_k. (a) Put $\Phi(C) = \sum_{(T,L)\in C} |T|^2$, a nonnegative integer, and take a cover C of F_k with $\Phi(C)$ least. (b) Suppose $(T, L) \in C$ is reducible, with T_1, T_2, L_1, L_2 as in the definition. By Proposition 8 there are kept blocks (T_i', L_i') with $T_i' \subseteq T_i$ and $L_i' \supseteq L_i$, $i = 1, 2$. Since $T_j \ne \emptyset$ for both j, $|T_i'| \le |T_i| < |T| \le k$, so both blocks lie in F_k. Let C' be C with (T, L) replaced by (T_1', L_1') and (T_2', L_2'). The sets T_1', T_2' are disjoint, being contained in the disjoint sets T_1, T_2, and are disjoint from the T-parts of the other blocks of C, which are disjoint from T. The union of the L-parts contains $L_1 \cup L_2 = L$, so it is still Low$'$. Thus C' is a cover of F_k, and

$$\Phi(C') - \Phi(C) = |T_1'|^2 + |T_2'|^2 - |T|^2 \le |T_1|^2 + |T_2|^2 - (|T_1| + |T_2|)^2 = -2|T_1||T_2| < 0,$$

contrary to the choice of C. (c) Hence no block of C is reducible, and C is a cover of R_k. Since $R_k \subseteq F_k$ and C minimises Φ over the covers of F_k, it also minimises Φ over the covers of R_k, which gives the statement on the least value of $\sum |T|^2$. □

The number of blocks is not preserved: for $m = 4$ and $k = 3$ (computed, block $(\emptyset, \emptyset)$ excluded) the least cover of F_k has 1 block and the least cover of R_k has 3. The nonemptiness of L_1, L_2 is not used. For a kept block it follows from that of T_1, T_2: if $L_1 = \emptyset$ then (T_2, L) is valid with $T_2 \subsetneq T$, contradicting the minimality of T. Computed check (COMPUTED): for each $2 \le m \le 17$ the program `st89` computes F_4 and R_4 from the brute-force enumeration above and decides by exhaustive search whether each has a cover; the answers agree (both have a cover for every m in this range), and for every block $B \in F_4$ the lows of B are covered by blocks of R_4 whose T-parts lie in the T-part of B (the local form of the replacement argument; 0 failures). At $m = 13$, $|F_4| = 4{,}478$ and $|R_4| = 322$ counting the block $(\emptyset, \emptyset)$, and 4,477 and 321 without it (the families of the programs). Reported effect on the search families (logged in development runs, not re-run for the archive; HEURISTIC): for $m = 49$, 1,992,263 blocks reduce to 26,427; for $m = 62$, 6.3×10^6 to 50,638. The reduction made the integer program tractable: before it, GLPK on the full family at $m = 47$ found no cover in about 12 minutes.

The block with empty T. A block $(\emptyset, L)$ is valid if and only if every element of L is a singleton minimal bad set: $\tau(L) \le |L|$, and $\tau(L) = |L|$ holds if and only if for each $l \in L$ the set $L \setminus \{l\}$ misses some minimal bad set contained in L, which must then be $\{l\}$. The minimal bad sets are nonempty: among the $2m+1$ points $0, t, \ldots, 2mt$ of the circle two lie within distance $1/(2m+1) < 1/(m+1)$, so for every t some $x \in [1, 2m]$ has $\|xt\| < 1/(m+1)$. The program `st89 s0` computes, for $2 \le m \le 64$, the velocities $x \in [1, 2m]$ for which $\{x\}$ is a bad set; they are 1 and $m+1$ only. So Low$'$ contains no singleton minimal bad set, and for these m the only kept block with $T = \emptyset$ is $(\emptyset, \emptyset)$ (a kept block $(T, \emptyset)$ with $T \neq \emptyset$ does not exist, since $(\emptyset, \emptyset)$ is valid). The block $(\emptyset, \emptyset)$ covers nothing, so removing it from a family changes neither the existence of covers nor the conclusion of Proposition 9. This is the family the search programs use. The check covers $2 \le m \le 64$ only. For the rows of Table 2 with $m \ge 66$ it was not repeated, so whether the family used there contains blocks with $T = \emptyset$ was not checked; this bears only on which covers the searches could find, since every certificate for $m \ge 66$ was accepted by the independent checker `v1` and the Lean kernel without reference to the family. The removal of $(\emptyset, \emptyset)$ itself loses no cover for any m, because that block has empty T and covers no element of Low$'$.

7 The verified values

Theorem 10 (VERIFIED for the rows of Tables 1 and 2)**.** *For every m in Table 1 or Table 2, and every set of m distinct integers in* $[1, 2m]$, ML $\ge 1/(m+1)$. *The Lean declaration is* ***`dense_floor_m`****m in the stated module. Together with Theorem 3 and Theorem 13, the dense window statement holds for every m with $2 \le m \le 126$. Its label is VERIFIED for every such m in the sense of Section 1: for each composite $2m+1$ a raw axiom print of the declaration is stored in the archive (file per row below).*

Bookkeeping. Composite $2m+1$ with $2 \le m \le 126$ occurs for 73 values of m (33 with $m \le 62$, 40 with $63 \le m \le 126$); 72 have a row (4 in Table 1, 68 in Table 2, of which 29 with $m \le 62$ and 39 with $m \ge 64$) and the 73rd is $m = 94$ (Section 8). These counts were confirmed for this version by `code/dense/check_rows.py` (output `data/dense/check_rows.out`: the 33 composite values with $m \le 62$ are covered by the rows and Table 1; of the 40 composite values in $[63, 126]$ exactly $m = 94$ has no row; 68 rows, 0 failures). Location of each archived print: $m = 4, 7, 10, 12, 13, 16, 17, 19, 22$: `dev_logs/thmD_axioms.out`; $m = 49, 52, 58$: `dev_logs/xc_lean_chain.log`; $m = 57$: `dev_logs/mk57.out`; $m = 59, 60, 61$: `dev_logs/mk59.out`, `mk60.out`, `mk61.out`; $m = 62$: `dev_logs/mk62.out` (monolithic; the split recheck is in `mkSA.out`); $m = 64$: `dev_logs/mkSB.out`, declaration `dense_floor_m64_split`; 38 values $66 \le m \le 126$ (66, 67, 70, 71, 72, 73, 76, 77, 79, 80, 82, 84, 85, 87, 88, 91, 92, 93, 97, 100, 101, 102, 103, 104, 106, 107, 108, 109, 110, 112, 115, 117, 118, 121, 122, 123, 124, 126): `data/dense/results.tsv`, split declarations; $m = 24, 25, 27, 28, 31, 32, 34, 37, 38, 40, 42, 43, 45, 46, 47, 55$: `data/dense/results_L16.tsv`; $m = 94$: `data/dense/m94_lean/ax_S94.out`. (All `dev_logs` paths are under `data/dense`. `results.tsv` has no row for $m = 64$.) The modules for $m \in \{24, 25, 27, 28, 31, 32, 34, 37, 38, 40, 42, 43, 45, 46, 47, 55\}$ and for $m \in \{97, 103, 109, 115, 122\}$ were built on 2026-11-03 under a memory guard (one module at a time; axiom print `propext, Classical.choice, Quot.sound`, zero `sorry`; for the five larger rows the certificates were also accepted by the independent checker `v1`; Lean build times 581, 266, 115, 125, 141 s for $m = 97, 103, 109, 115, 122$ and 5 to 456 s for the sixteen others, in the two results files). Their declarations are `dense_floor_m`m for the sixteen and `dense_floor_m`m`_split` for the five. For $m = 94$ no cover-capacity certificate of Theorem 5 is known: a search of three hours (`code/`

dense/src/m94, log in data/dense/m94) ended without a cover, and its exact phases stopped when they reached a preset limit on the number of search nodes, so that search decided nothing; it was a single logged run that was not repeated. The case $m = 94$ is closed in Section 8 by a different route. (The case $m = 1$ is elementary and not formalized here.) For $m \geq 64$ the declaration is `dense_floor_m`m`_split` (split form, Remark 11) except $m = 94$; for $m \leq 62$ it is `dense_floor_m`m. The statement for $m \leq 62$ is the union of Theorem 3 (all primes $2m + 1$) and the 33 composite values of $2m + 1$ with $m \leq 62$, namely $m = 4, 7, 12, 13$ (Table 1) and the 29 rows of Table 2 with $m \leq 62$ (the first of which is $m = 10$). For $63 \leq m \leq 126$ there are 40 values with $2m + 1$ composite, 39 of which are rows of Table 2; the other one is $m = 94$. The script code/dense/check_rows.py regenerates the parts, largest part and witness-time counts of the rows from the Lean modules shipped in lean/, has the independent checker v1 accept each certificate (68 of 68 ACCEPT in the run for this version), and compares the lists of composite m. Before this version the composite $m \in [63, 126]$ without a row were $\{94, 97, 103, 109, 115, 122\}$; the five rows $m = 97, 103, 109, 115, 122$ are now in the table.

Notes to Table 2. *Contents.* Cover-capacity certificates, 68 rows ($m = 10, \ldots, 62$: 29 rows; $m = 64, \ldots, 126$: 39 rows; the four rows $m = 4, 7, 12, 13$ of Table 1 are not repeated). Parts: number of parts U_i; max: largest part; times: number of witness times. In every row $\sum c_i = m-1$. Modules carry the prefix Delta2. For $m \geq 64$ the modules are in split form (Remark 11); the row $m = 64$ is module CoverRangeSB, and $m = 62$ was additionally rechecked in split form as CoverRangeSA, same certificate. In these rows, times is the number of witness times in the module.

Build times. Build times and axiom prints for $m \geq 66$: data/dense/results.tsv (38 rows: the 33 rows of version 1.0.4 and the five rows $m = 97, 103, 109, 115, 122$ built on 2026-11-03): 45 to 176 s per module for the 33 rows $m \geq 66$ other than the five, 115 to 581 s for those five (581 s for $m = 97$, 266 s for $m = 103$), 56 s for $m = 64$ (print in dev_logs/mkSB.out; results.tsv has no row for $m = 64$). The sixteen rows $m \leq 55$ marked in the paragraph on labels were built on 2026-11-03 in 5 to 456 s each (456 s for $m = 37$, 364 s for $m = 43$; data/dense/results_L16.tsv); these are single runs with the dependencies already built, so a short time does not include building them.

Search column (provenance only: a certificate is accepted or rejected by the checkers independently of how it was found; the entries, including the times, are single logged runs of the development pipeline, HEURISTIC as statements about the searches, and were not re-run). "Integer program": an integer program of the earlier pipeline whose solver and time are not recorded. "Pool search + MILP": selection from a pool of candidate parts by a mixed-integer solver (HiGHS); "n/r": time not recorded. "Column generation": a Rust column generation whose master linear program is solved by HiGHS through Python. "GLPK": the GNU Linear Programming Kit solver run on a Rust-written model over the irreducible block family of Proposition 9. "Local search, checked": a Rust local search (program g1) followed by acceptance by an independent Rust checker (program v1, Section 9). Driver output of the column generation and GLPK runs for $m \geq 27$ is in data/dense/dev_logs; no log was located for the integer program rows.

Axioms column. The mark S is defined in Section 1: the print of the axioms of the declaration of the row (`dense_floor_m`m, or `dense_floor_m`m`_split` for $m \geq 64$) equals propext, Classical.choice, Quot.sound, stored as raw output in the archive, in these files under data/dense: dev_logs/thmD_axioms.out for $m = 10, 16, 17, 19, 22$; dev_logs/xc_lean_chain.log for $m = 49, 52, 58$; dev_logs/mk57.out, mk59.out, mk60.out, mk61.out, mk62.out for $m = 57, 59, 60, 61, 62$; dev_logs/mkSB.out for $m = 64$ (split declaration); results.tsv for the

38 rows $66 \le m \le 126$, among them the five rows $m = 97, 103, 109, 115, 122$ (print observed in a development build of 2026-11-03); `results_L16.tsv` for the sixteen rows $m = 24, 25, 27, 28, 31, 32, 34, 37, 38, 40, 42, 43, 45, 46, 47, 55$ (also built on 2026-11-03). In the lines of `results_L16.tsv` the first message is an error of the print script, which asked for a constant named `dense_floor_m`m`_split` that does not exist for these monolithic modules; the print of `dense_floor_m`m follows it, and a path inside the error message was rewritten to `<dev-tree>`. A row without a located print would carry no mark; there is none. The prints in `dev_logs` cover `dense_floor_of_prime` and $m = 4, 7, 10, 12, 13, 16, 17, 19, 22$ (one Lean process on 2026-10-01 after building `Delta2Pigeonhole`, `Delta2DigitGreedy` and `Delta2DenseWindow`), and $m = 49, 52, 57, 58, 59, 60, 61, 62, 64$. No Lean build was run in preparing this version of the paper; every build time and axiom print quoted is from an earlier development run.

Table 2: Cover-capacity certificates, 68 rows, $m = 10$ to $m = 126$ except 94 and the four m of Table 1; composite $2m + 1$ only. Mark S as in Section 1 (all 68 rows). Notes to the columns are in the paragraph before this table.

m	$2m+1$	parts	max	times	search	module	axioms
10	$3 \cdot 7$	5	6	14	integer program	`CoverCapacity`	S
16	$3 \cdot 11$	7	6	22	integer program	`CoverCapacity`	S
17	$5 \cdot 7$	6	9	26	integer program	`CoverCapacity`	S
19	$3 \cdot 13$	6	9	27	integer program	`CoverCapacity`	S
22	$3^2 \cdot 5$	7	8	31	integer program	`CoverCapacity`	S
24	7^2	10	6	33	integer program	`CoverCapacity`	S
25	$3 \cdot 17$	12	8	43	pool search + MILP, ∼7 min	`CoverRangeI`	S
27	$5 \cdot 11$	17	8	41	pool search + MILP, n/r	`CoverRangeJ`	S
28	$3 \cdot 19$	13	8	48	pool search + MILP, ∼17 min	`CoverRangeK`	S
31	$3^2 \cdot 7$	17	12	51	column generation, 1 s	`CoverRangeM`	S
32	$5 \cdot 13$	20	8	45	pool search + MILP, 635 s	`CoverRangeL`	S
34	$3 \cdot 23$	18	12	54	column generation, 62 s	`CoverRangeN`	S
37	$3 \cdot 5^2$	16	13	62	column generation, 358 s	`CoverRangeO`	S
38	$7 \cdot 11$	23	10	55	column generation, 4 s	`CoverRangeP`	S
40	3^4	22	10	61	column generation, 35 s	`CoverRangeQ`	S
42	$5 \cdot 17$	25	10	61	column generation, 10 s	`CoverRangeR`	S
43	$3 \cdot 29$	21	12	73	column generation, 118 s	`CoverRangeS`	S
45	$7 \cdot 13$	28	8	64	column generation, 5 s	`CoverRangeT`	S
46	$3 \cdot 31$	22	10	74	column generation, 172 s	`CoverRangeU`	S
47	$5 \cdot 19$	26	10	67	column generation, 32 s	`CoverRangeV`	S
49	$3^2 \cdot 11$	26	9	77	GLPK, 24.6 s	`CoverRangeAD`	S
52	$3 \cdot 5 \cdot 7$	23	9	85	GLPK, 74.1 s	`CoverRangeAE`	S
55	$3 \cdot 37$	25	11	90	column generation, 521 s	`CoverRangeW`	S
57	$5 \cdot 23$	38	10	79	column generation, 4 s	`CoverRangeX`	S
58	$3^2 \cdot 13$	25	9	97	GLPK, 6.3 s	`CoverRangeAF`	S
59	$7 \cdot 17$	29	10	88	column generation, 100 s	`CoverRangeZ`	S
60	11^2	38	10	84	column generation, 8 s	`CoverRangeAA`	S
61	$3 \cdot 41$	26	11	93	column generation, 875 s	`CoverRangeAB`	S

m	$2m+1$	parts	max	times	search	module	axioms
62	5^3	44	8	81	GLPK (weighted), 15 min	`CoverRangeAC`	S
64	$3 \cdot 43$	23	9	108	local search, checked	`CoverRangeSB`	S
66	$7 \cdot 19$	25	9	112	local search, checked	`CoverRangeS66`	S
67	$3^3 \cdot 5$	26	9	110	local search, checked	`CoverRangeS67`	S
70	$3 \cdot 47$	25	9	118	local search, checked	`CoverRangeS70`	S
71	$11 \cdot 13$	26	9	120	local search, checked	`CoverRangeS71`	S
72	$5 \cdot 29$	26	9	122	local search, checked	`CoverRangeS72`	S
73	$3 \cdot 7^2$	29	8	120	local search, checked	`CoverRangeS73`	S
76	$3^2 \cdot 17$	32	8	121	local search, checked	`CoverRangeS76`	S
77	$5 \cdot 31$	27	9	130	local search, checked	`CoverRangeS77`	S
79	$3 \cdot 53$	36	9	123	local search, checked	`CoverRangeS79`	S
80	$7 \cdot 23$	28	9	135	local search, checked	`CoverRangeS80`	S
82	$3 \cdot 5 \cdot 11$	38	8	129	local search, checked	`CoverRangeS82`	S
84	13^2	27	9	146	local search, checked	`CoverRangeS84`	S
85	$3^2 \cdot 19$	35	9	138	local search, checked	`CoverRangeS85`	S
87	$5^2 \cdot 7$	31	9	147	local search, checked	`CoverRangeS87`	S
88	$3 \cdot 59$	32	9	147	local search, checked	`CoverRangeS88`	S
91	$3 \cdot 61$	36	8	149	local search, checked	`CoverRangeS91`	S
92	$5 \cdot 37$	37	9	151	local search, checked	`CoverRangeS92`	S
93	$11 \cdot 17$	31	9	160	local search, checked	`CoverRangeS93`	S
97	$3 \cdot 5 \cdot 13$	33	11	175	local search, checked	`CoverRangeS97`	S
100	$3 \cdot 67$	39	9	171	local search, checked	`CoverRangeS100`	S
101	$7 \cdot 29$	37	9	169	local search, checked	`CoverRangeS101`	S
102	$5 \cdot 41$	38	9	173	local search, checked	`CoverRangeS102`	S
103	$3^2 \cdot 23$	38	10	174	local search, checked	`CoverRangeS103`	S
104	$11 \cdot 19$	39	9	174	local search, checked	`CoverRangeS104`	S
106	$3 \cdot 71$	38	8	179	local search, checked	`CoverRangeS106`	S
107	$5 \cdot 43$	37	9	183	local search, checked	`CoverRangeS107`	S
108	$7 \cdot 31$	36	9	183	local search, checked	`CoverRangeS108`	S
109	$3 \cdot 73$	42	9	182	local search, checked	`CoverRangeS109`	S
110	$13 \cdot 17$	38	9	187	local search, checked	`CoverRangeS110`	S
112	$3^2 \cdot 5^2$	42	9	186	local search, checked	`CoverRangeS112`	S
115	$3 \cdot 7 \cdot 11$	46	9	187	local search, checked	`CoverRangeS115`	S
117	$5 \cdot 47$	43	9	196	local search, checked	`CoverRangeS117`	S
118	$3 \cdot 79$	43	9	199	local search, checked	`CoverRangeS118`	S
121	3^5	47	9	203	local search, checked	`CoverRangeS121`	S
122	$5 \cdot 7^2$	47	9	202	local search, checked	`CoverRangeS122`	S
123	$13 \cdot 19$	40	9	214	local search, checked	`CoverRangeS123`	S
124	$3 \cdot 83$	48	9	204	local search, checked	`CoverRangeS124`	S
126	$11 \cdot 23$	41	9	216	local search, checked	`CoverRangeS126`	S

Notes on the search record. The timings, node counts and seeds in these notes are single logged runs of the development pipeline; none was re-run for the archive, and as statements about the searches they are HEURISTIC. The certificates themselves do not depend on them. (i) The

earlier pool-based pipeline used 64-bit masks. For $m \geq 33$ the shifts $1 \ll (x-1)$ wrap modulo 64, so every "infeasible" reported by that pipeline for $m \in \{34, 37, 38, 40, 42, 43, 45, 46, 55, 57, 58, 59\}$ is void. The certificates of rows $m \leq 32$ are unaffected, since each was re-checked by exact capacity and then proved in Lean. (ii) The certificates for $m = 49, 52, 58$ came from GLPK (the binary `glpsol`) run on a model written by Rust; the Rust depth-first search over the same family did not find them within the time spent ($m = 49$: about 6.5×10^7 nodes in about 1400 s) and decided nothing there. (iii) The column generation search used the open-source solver HiGHS, driven from Python, for the master linear program and the integer step. (iv) Earlier failures at $m = 49, 52, 58$ were limits of the column pool, not properties of the method: all three have covers with $|T| \leq 4$. (v) A pure Rust local search (weighted, keeping the blocks disjoint on Y, on the irreducible $|T| \leq 4$ family) found covers for $m = 46, 49, 52, 58, 62$ without any external solver, in 4 to 44 seconds on two seeds (one $m = 52$ run among three extra seeds found nothing within 300 s, so the search is heavy-tailed there). Each cover passed the exact capacity check. These covers are COMPUTED and have not been built in Lean; the Lean modules in Table 2 use the earlier certificates. A local search cannot decide non-existence, and no non-existence claim is made.

Remark 11 (Split form: chunked checks, formalization device). In the split-form modules ($m = 62$ recheck and $m \geq 64$), the finite checks demanded by Theorem 5 are stated as separate lemmas, each closed by the Lean tactic `decide` with the option `+kernel`, which evaluates the decidable proposition by reduction in the Lean kernel without compiled code: one lemma each for the denominators, the cover and the capacity sum, windows of four witness times for the bad-set inclusions, and one lemma per part for the capacity condition, assembled by case distinction on the index. This is a device for keeping elaboration memory bounded. It changes neither the statements nor the certificates (the $m = 62$ certificate is the same in the monolithic module and in its split recheck) and carries no mathematical content. The data are plain lists; no `native_decide` and no `sorry` occur.

8 The value $m = 94$

This section proves the dense window statement for $m = 94$ ($2m + 1 = 189 = 3^3 \cdot 7$, window $[1, 188]$, floor $1/95$), the one composite $2m+1 \leq 253$ without a row in Table 2. It uses Theorem 5 in a pinned form (Proposition 12) and three certificates of that form (Table 3); of the earlier sections it uses only the bad sets $B(p/q)$ and Lemma 1. The exhaustive computation of the first paragraph below is corroboration and is not used by the Lean proof.

Hitting-set form (reduction PROVED in the review reports; the computation COMPUTED). Let $W_0 = 188$ be the height of the window. If $\mathrm{ML}(v) \geq 1/95$ for some v then the maximum of gap_v is attained at a time k/s with $s \leq 2W_0 = 376$ and $0 < k/s \leq 1/2$: at a maximiser t^* with value $\mu^* > 0$ either $\mu^* = 1/2$ and some $v_i t^*$ is a half-integer, so $t^* = (2a+1)/(2v_i)$, or each active term $\|v_i t\|$ has local slope $\pm v_i$, there are active terms of both signs (otherwise gap_v would increase on one side of t^*), and for active v_i, v_j of opposite slope $v_i t^* = a + \mu^*$, $v_j t^* = b - \mu^*$, so $(v_i + v_j)t^* \in \mathbb{Z}$. Hence a set V of at most 94 elements of $[1, 188]$ has $\mathrm{ML}(V) < 1/95$ if and only if it meets every bad set $B(k/s)$ with $s \leq 376$, that is, every member of the family of minimal bad sets. At $m = 94$ this family has 3,724 members, of sizes 1 to 4, coming from the 21,511 reduced fractions k/s with $s \leq 376$, $k \leq s/2$. The corresponding families for the windows of height $2n$ have 55, 163, 402, 1,607 and 3,724 members for $n = 12, 20, 31, 62, 94$; for $n = 31, 62, 94$ they were produced by two independent programs and agree as sets of sets (reports of two review sessions).

So the statement for $m = 94$ is equivalent to: this hypergraph has no transversal of size 94 (or less).

Exhaustive computation (COMPUTED, corroboration). A branch-and-bound over transversals in exact integer arithmetic (192-bit masks, forced moves, domination) exhausted its search tree after 11,086,881 nodes (229 s on one core) without finding a transversal of size 94; the same search with a second branching order took 7,754,163 nodes and with unit propagation and domination 9,972,053 nodes, with the same outcome. A separate replay program, sharing no code with the solver, re-derived every edge from its witness time and accepted the logged tree. A second solver written in a review session gave the same outcome in 4,711,079 nodes, and the cadical solver, given the 3,724 clauses and a totalizer encoding of the bound 94, answered unsatisfiable in 781 s (no proof certificate was produced at $m = 94$; a DRAT proof was checked at $n = 62$ only). Controls: with bound 95, and with the threshold 1/94 in place of 1/95, the search finds a transversal; deleting the edge $\{1\}$ or $\{95\}$ gives a false counterexample, as it must. The programs and logs of these runs are not in this archive; the figures are those of the run report and of two review reports, and none was repeated for this release. This computation concerns sets of at most 94 speeds (multisets), whereas the Lean theorem below is stated for 94 distinct speeds.

Proposition 12 (VERIFIED)**.** *Let $k+1 = m$ and let $v : \mathrm{Fin}(k+1) \to \mathbb{Z}$ be positive and injective with values in $[1, 2m]$. Let finitely many times (p_j, q_j) and sets $\mathtt{bad}_j$ be as in Theorem 5. Let $I, O \subseteq [1, 2m]$ be such that no element of I is a speed and every element of O is a speed. Let parts $U_1, \ldots, U_t$ cover $[1, 2m] \setminus (I \cup O)$ and let c_i be such that every $T \subseteq U_i$ with $|T| > c_i$ contains $\mathtt{bad}_j \setminus I$ for some j. If $\sum_i c_i + |I| < m$ then $\mathrm{ML}(v) \geq 1/(m+1)$.*

Lean: `floor_of_pinned_cover` (module `Delta2PinnedCover`). The case $I = O = \emptyset$ is Theorem 5 (declaration `floor_of_cover_capacity_of_pinned`, same module).

Proof. Suppose $\mathrm{ML}(v) < 1/(m+1)$ and let V be the set of speeds and $W = [1, 2m] \setminus V$, so $|W| = m$ because v is injective with values in the window. Every time p_j/q_j is defeated (Lemma 1), so every $\mathtt{bad}_j$ meets V. The elements of I are not speeds, so $I \subseteq W$; put $W' = W \setminus I$. If $|W' \cap U_i| > c_i$, some $\mathtt{bad}_j \setminus I$ lies in $W' \cap U_i$; a point x of $\mathtt{bad}_j$ in V is not in I, hence lies in $\mathtt{bad}_j \setminus I \subseteq W$, which contradicts $x \in V$. So $|W' \cap U_i| \leq c_i$. The elements of O are speeds, so W' avoids $I \cup O$ and lies in $\bigcup_i U_i$. Therefore $m = |W| \leq |I| + |W'| \leq |I| + \sum_i c_i < m$, a contradiction. □

Branching. Let $a, b \in [1, 2m]$. A set of speeds falls in exactly one of three cases: a is not a speed (case B, pins $I = \{a\}$, $O = \emptyset$); a is a speed and b is not (case C, $I = \{b\}$, $O = \{a\}$); a and b are speeds (case A, $I = \emptyset$, $O = \{a, b\}$). Lean: `pinned_tree_three`, which also carries the order in which the two hypotheses are passed.

The three leaves for $m = 94$. Take $a = 67$, $b = 75$. The certificate of case A is a partition of $[1, 188] \setminus \{67, 75\}$ with $\sum_i c_i = 93$ a saved near-solution of the development repository's local search for a cover of the window, in which exactly two speeds, here 67 and 75, remain uncovered at cost 93 (state 416 of the saved pool, which contained 24 states with two uncovered speeds; the pool is not archived). The certificates of cases B and C, with capacities summing to 92, were found by local search warm-started from that partition and were accepted by an independent checker; for each case the budget $\sum_i c_i + |I| = 93 < 94$ holds (Table 3).

Theorem 13 (VERIFIED)**.** *Every set of 94 distinct integers in $[1, 188]$ satisfies $\mathrm{ML} \geq 1/95$.*

leaf	case	I	O	parts	times	$\sum c_i + \lvert I\rvert$	module
A	67, 75 speeds	$\emptyset$	$\{67, 75\}$	28	167	$93 + 0$	`CoverRangeS94s416A`
B	67 not a speed	$\{67\}$	$\emptyset$	33	162	$92 + 1$	`CoverRangeS94s416B`
C	67 speed, 75 not	$\{75\}$	$\{67\}$	34	160	$92 + 1$	`CoverRangeS94s416C`

Table 3: Pinned cover-capacity leaves for $m = 94$ (all three rows marked S). Modules carry the prefix `Delta2`; the leaf theorems are `leaf94_s416A`, `leaf94_s416B`, `leaf94_s416C`. Certificates: `data/dense/m94_lean/certs`. Parts and times are the numbers of parts U_i and of witness times in the module.

Lean: `dense_floor_m94` (module `Delta2CoverRangeS94`) states, for $v : \mathrm{Fin}\, 94 \to \mathbb{Z}$ positive, injective on `toNat` and with `toNat` in $[1, 188]$, that $1/95 \le \mathrm{ML}(v)$. The proof is `pinned_tree_three 67 75` applied to the three leaf theorems, each an instance of `floor_of_pinned_cover` whose finite checks are lemmas closed by `decide +kernel` (273 in the five modules, by the count of the reviewing session below). Each leaf module has the split form of Remark 11.

Proof. Case B: 67 is not a speed; leaf B applies Proposition 12 with $I = \{67\}$, $O = \emptyset$. Case C: 67 is a speed and 75 is not; leaf C, $I = \{75\}$, $O = \{67\}$. Case A: both are speeds; leaf A, $I = \emptyset$, $O = \{67, 75\}$. The cases are exhaustive, and in each the budget is below 94. □

Builds and prints (carried over). On 2026-11-08, in the development repository, one module at a time under a memory guard: `Delta2PinnedCover` 36 s, peak group memory 5.6 GB; leaf A 371 s, 8.2 GB (an earlier attempt failed on a syntax error in the text of the leaf theorem, not in a finite check, and the generator was corrected); leaf B 237 s, 8.7 GB after two builds of a version with coarser chunking of the witness checks were stopped by the 9 GB guard, and a rebuild with two witnesses per lemma succeeded on the same data (whether chunking or run variance caused the stops was not established); leaf C 255 s, 8.8 GB; final module 27 s. The print of the axioms of `dense_floor_m94`, `floor_of_pinned_cover` and of each leaf theorem is `propext, Classical.choice, Quot.sound`; that of `pinned_tree_three` is `propext, Quot.sound` (`data/dense/m94_lean/ax_S94.out`, `axioms_fresh_review.out`). A search for `sorry`, `native_decide` and `axiom` in the five modules found nothing.

Review and limits. A fresh session of the AI system, working from the Lean files before reading the build report, found no defect in the statement, the pin logic, the wiring of the leaves, or the budgets. It checked that every hypothesis of the leaf theorems is used, that the statement is satisfiable and tight on its own instance ($v = (1, \ldots, 94)$, ML $= 1/95$ by the exact routine), that the data of each leaf module equal the certificate file, and by a separate brute-force program that every witness-inclusion and every capacity condition holds in each leaf. It did not recompile the modules from scratch: it observed that the build is up to date and printed the axioms. The Lean statement was not reviewed by a human. The generator `code/dense/src/gen94` regenerates the three leaf modules from the three certificate files byte for byte (run for this release: `data/dense/m94_lean/regen_check.out`). Only the state 416 was built in Lean; certificate files for other states exist in the development tree and are not used.

9 Reproducibility

All paths are relative to the root of the accompanying archive; `README.md` there gives the toolchain. The Rust tools are built by `bash code/dense/build.sh`. The program names used in the text are directories of `code/dense/src`, each a Rust program with exact integer arithmetic: `thmd` (the exact checks of Section 3); `fb` (checker of Table 1); `yind` (test of Proposition 6); `st89` (brute-force test of Propositions 8 and 9); `xc` (enumerator of the block family of Section 6); `pool` and `cg` (the earlier pool search and column generation, each with an exact checker and Lean emitter); `g1` (local search for covers); `v1` (independent checker of a cover-capacity certificate, written from the statement of Theorem 5); `lg` (Lean module generator, command `lg2`); `examples` (the numbers of Section 5). For $m \geq 64$ the pipeline is: `g1` (Rust local search, source `code/dense/src/g1`) writes `data/dense/certs/mM.g1.sol`; `v1 check M mM.g1.sol` (Rust, source `code/dense/src/v1`, no shared code with the search) must print ACCEPT; `lg2 gen2 M mM.g1.sol S<M> out` (source `code/dense/src/lg`) emits the Lean module; the modules are built one at a time and the axioms recorded in `data/dense/results.tsv` (for $m \geq 66$; for $m = 64$ in `data/dense/dev_logs/mkSB.out`). The split form was introduced because the Lean build of the first module for $m = 64$, written in the monolithic form of the earlier rows, was killed by the operating system for memory after 282 s (exit 137); the $m = 62$ certificate built in 43 s in split form (monolithic: 130 s), and $m = 64$ in 56 s.

After the build, two scripts regenerate checkable content of Table 2 from the archive alone.

```
bash code/dense/build.sh build/dense
python3 code/dense/check_rows.py build/dense/v1
bash code/dense/regen_lean.sh build/dense
```

The second command reads the certificate (parts and capacities) out of each shipped Lean module, recomputes the columns parts, max and times, has `v1` accept the certificate independently, and compares the composite lists. The third regenerates the Lean modules for $m \geq 64$ from the certificates and compares them with the shipped modules. Both check exactly the data that the Lean kernel accepted. The output of a run is `data/dense/check_rows.out`. Two further programs regenerate content of the text. `code/dense/src/fb` regenerates Table 1 from the Lean source, and `code/dense/src/yind` regenerates the first sentence of Proposition 6 for $2 \leq m \leq 62$:

```
build/dense/fb check lean/LRSpectrumFull/Delta2FracBlocking.lean
build/dense/fb selftest lean/LRSpectrumFull/Delta2FracBlocking.lean
build/dense/yind 2 62
build/dense/yind control 13
```

What is not regenerated by a script in the archive: the search-method and time column for the rows $m \leq 62$ and the build times, which are logged values of single runs (partly archived as raw logs in `data/dense/dev_logs`); the build times and axiom prints of all rows (stored logs of development builds; no script of the archive runs Lean); the programs and logs of the exhaustive computation for $m = 94$ reported in Section 8 (not archived); and the three-hour search for $m = 94$ (log archived, not repeated). The searches of the earlier pipeline are in `code/dense/src` (`pool`, `cg`, `xc`); their commands are

```
bin/cg run M 1 10 900 60 6000     # seed smax total_s mip_s attempts
bin/cg check M certs/mM.sol out.lean
bin/xc blocks M 4 6
```

```
bin/xc reduce M 4
IRR=1 code/dense/run_glp.sh M 4 2400 -99
bin/xc selftest M 4               # M <= 17
```

run from `data/dense`. Lean: one module at a time, `cd lean && lake build LRSpectrumFull.Delta2CoverRange<LETTER>`; axioms by `#print axioms LRSpectrumFull.dense_floor_m<M>` in a file with the matching `import`. A bare whole-tree `lake build` exhausts memory on a machine with 24 GB. Toolchain: Lean 4.30.0, Lake 5.0.0, Mathlib as pinned in `lean/lake-manifest.json`.

The value $m = 94$. `bash code/dense/build.sh build/dense` also builds `gen94`. For each leaf, `build/dense/gen94 gen data/dense/m94_lean/certs/<cert> <S94s416A|B|C> out.lean <J>` (with the certificate files `leafA_state416.cert`, `leaf_s416B67.cert`, `leaf_s416C67.cert` and $J = 4, 2, 4$ witnesses per lemma) regenerates the module, and `gen94 verify <module>` re-parses an emitted module and re-checks its data in exact integers. The output of the run for this release is `data/dense/m94_lean/regen_check.out`: the three regenerated modules are identical to the shipped ones. Lean: `lake build LRSpectrumFull.Delta2CoverRangeS94` builds the five modules of Section 8 (not run for this release).

10 What is not claimed

- No new number of runners for the lonely runner conjecture. The window has height $2m$; the known reductions of the conjecture to finite checking require speeds of exponential size, so a statement at height $2m$ is a necessary special case at most.
- Nothing for m outside Theorem 3, Tables 1 and 2, and Theorem 13. No statement is made for $m \geq 127$ unless $2m + 1$ is prime (Theorem 3). For $m \leq 126$ with composite $2m + 1$ the statement is VERIFIED in the sense of Section 1: by the 72 rows (mark S) and, for $m = 94$, by the pinned tree of Section 8. The Lean theorems concern m distinct speeds in $[1, 2m]$. The form for a multiset of m speeds reduces to it by padding to m distinct speeds and by two lemmas of the development repository (`ML_dedup`, in module `Delta2MLScale`, which is not in the archive, and `ML_comp_le`); this reduction is PROVED on paper here and is not formalized for $m = 94$ nor uniformly in m. Theorem 3 has a weaker label than the rows (one archived print of 2026-10-01, no rebuild, statement matched by reading). No Lean build was run to prepare this release. The Lean sources in the archive are provided as is: the archived tree was not built as a whole (a module that imports all the range modules needs more than 20 GB of memory, so the modules are built one at a time), and not every module of the development repository is in the archive.
- Theorem 3 is a short consequence of known ingredients (Tao's Lemma 5.1(i), the Bohman–Peng witness device, and one forcing step). It gives the window $[1, 2m]$ for all m with $2m+1$ prime, a set of density zero, and does not settle Problem 6 of [7]: nothing is proved here for composite $2m + 1$ with $m \geq 127$, and the question of [12] remains open for those m.
- A failed search is not evidence that no certificate exists (Section 7, notes (ii) and (iv)).
- For $m \leq 12$ the statement is contained in the computer-assisted results claimed for the lonely runner conjecture for up to 12 speeds; for $m = 13, 14$ by the preprint [1] (version 2),

which is not refereed (Section 11). For $m \geq 16$ with $2m+1$ composite no earlier statement was found, which is a search result and not a proof of novelty.

11 Prior art

Statements below were obtained by reading arXiv texts and abstract pages and by web searches; no Google Scholar or MathSciNet access was available.

Window and prime case. Tao [13] states (as described in his blog post) the floor for speeds in $[1, 1.2m]$, proved in the accompanying preprint [12], and remarks that $[1, 2m]$ is a natural target; Lemma 5.1(i) gives the divisibility statement used in Lemma 1. Bohman and Peng [5] prove, for m large, the strict inequality $1/(m+1) < \{v_i t\} < m/(m+1)$ for some t when $m < v_m \leq 2m - \exp(C(\log\log m)^2)$. In their proof, if s, t are coprime non-speeds with $2m - 2k < s + t \leq 2m$ and $q = s^{-1} \bmod (s+t)$, then the time $q/(s+t)$ is a witness. The argument uses only that s, t are not speeds, that $\gcd(s, s+t) = 1$, and that $s+t$ exceeds the largest speed; it therefore applies verbatim at $s + t = 2m + 1$, where for $2m+1$ prime coprimality is automatic. Bohman and Peng do not state this case (their hypothesis is $s + t \leq 2m$) and do not count multipliers; Lemma 4 obtains the pair of non-speeds $s, N - s$ by counting classes modulo N from the pair of speeds that Lemma 3 supplies. Pomerance [8] improves the margin in the result of Bohman and Peng (his Theorem 1, on coprime matchings between intervals) and states in the introduction that the interval $[1, 2m]$ remains a challenge; Pandey [6] treats speeds in arithmetic progressions and excludes exactly the progression with first term and difference both equal to 1. Only the arXiv version 1 of [5] was read; the published Mathematika text could not be read (HTTP 403 on the publisher site), Crossref lists no erratum, and a later remark in the published text could change the verdict on novelty.

Small m. The conjecture for up to 12 speeds (13 runners) is claimed in [9, 14, 10, 11], and for 13 and 14 speeds (14 and 15 runners) in [1], version 2 of 24 September 2026, a computer-assisted preprint that is not refereed and whose computation has not been reproduced here. This subsumes $m \leq 12$, and $m = 13, 14$ conditionally on [1]. The first m with $2m+1$ prime outside the range $m \leq 12$ is $m = 14$ ($2m + 1 = 29$); outside the range $m \leq 14$ it is $m = 15$.

Zhang [15] classifies the tight one-entry modifications of $\{1, \ldots, m\}$; the paper is related work and is not used here.

Method. The integral covering formulation of witness times (a tuple has no witness iff every allowed time is ruled out by some speed) is used computationally in [9, 14, 10, 11, 1]. The closest use of a weighting of times found is Bedert's Lemma 2.1 [3] (a finitely supported measure tested against a family), of which the weights of Proposition 4 are an instance; no use of capacity cuts or of integer covers with capacities was found for the lonely runner problem; the duality itself is textbook, so the only possible novelty is the application. Status: HEURISTIC (absence of evidence from the searches recorded in the audit).

Search of 2026-11-09. A further search for the statement of Theorem 3 and of the window $[1, 2m]$ was made on 2026-11-09 (web search and arXiv abstract pages; full text read for Sungkawichai and Trakulthongchai [11] and for Beck and Everett [2]; the blog post of Tao [13] read in part, from which the statements on $[1, 1.2m]$ and $[1, 2m]$ are taken; those statements were

not checked in the preprint [12]). It covered Tao [12], the survey of Perarnau and Serra [7], Bedert [3], Sungkawichai and Trakulthongchai [11], Zhang [15], Beck and Everett [2], Rosenfeld [9] and Blanco, Criado and Santos [4]. It found no paper that proves the lonely runner inequality for all sets of m distinct speeds in $[1, 2m]$, either for all m or for an infinite family of m. Proposition 1.4 of [11] (a polynomial-method statement for speeds congruent to i modulo a large prime p, for $k+1$ and $p > k^2 + k$ odd primes) is a statement of a different kind: it prescribes residues, whereas the window prescribes the size of the speeds. Not read in full text in that search: the papers of Bohman and Peng [5], Pomerance [8] and Pandey [6], and the finite-checking paper of Malikiosis, Santos and Schymura; the remaining prior-art risk lies there, and the paragraph on the window and the prime case above rests on the earlier reading of October 2026 (arXiv version 1 only for [5]), which has not been re-checked against the present statements. The Lean formalization of an LRC statement was also looked for: no Lean or Isabelle formalization of the computer-assisted cases (8 to 15 runners) was found in that search. A search that finds nothing is not a proof that nothing exists.

References

[1] Allikvere. Fourteen and fifteen lonely runners, 2026. arXiv:2609.02604 (v2); not refereed as far as known.

[2] Beck and Everett. Lonely runner relations, 2026. arXiv:2609.06259.

[3] Bedert. Riesz products and the lonely runner conjecture: A wider gap of loneliness, 2025. arXiv:2511.16636.

[4] Blanco, Criado, and Santos. Coloopless zonotopes and counterexamples to the shifted lonely runner conjecture. *Rev. Mat. Complut.*, 2026. arXiv:2603.24784; accepted.

[5] Tom Bohman and Fei Peng. Coprime mappings and lonely runners. *Mathematika*, 68:784–804, 2022. arXiv:2109.09860 (v1 only).

[6] Pandey, 2010. Bibliographic details not verified; see the text for the statement used.

[7] G. Perarnau and O. Serra. The lonely runner conjecture turns 60. *Comput. Sci. Rev.*, 58, 2025. Article 100798; arXiv:2409.20160.

[8] Pomerance, 2022. Integers 22, Article A2; title not verified.

[9] Matthieu Rosenfeld. The lonely runner conjecture holds for eight runners, 2025. arXiv:2509.14111.

[10] Matthieu Rosenfeld. The lonely runner conjecture holds for nine runners, 2025. arXiv:2512.01912.

[11] Sungkawichai and Trakulthongchai. Eleven, twelve, and thirteen lonely runners, 2026. arXiv:2604.23906.

[12] Terence Tao. Some remarks on the lonely runner conjecture. arXiv:1701.02048.

[13] Terence Tao. Some remarks on the lonely runner conjecture. Blog post, `https://terrytao.wordpress.com/2017/01/10/some-remarks-on-the-lonely-runner-conjecture/`, 2017.

[14] Trakulthongchai. Nine and ten lonely runners, 2025. arXiv:2511.22427.

[15] Zhang. Tight instances of the lonely runner conjecture: one-entry modifications and the growth of speeds, 2026. arXiv:2608.13599.